CSC215 Lab7 – Assembly Language

Total: 100pt

@Copyright Dr. Xiang Fu. All rights reserved.

What to turn in: Submit **lab7.zip**, which contains the following:

(1) CSC215Lab7.docx
(2) cat.cc

Note: the files needed for this lab are located in ~/Desktop/csc215/lectures/lec16_assembly_practice in the Ubuntu image.

In the **lec16_assembly_practice** folder, you can find a simple C program which executes "/bin/ls" as an external program. The source code is shown as below:

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/types.h>


4 int main(){
5   char *args[] = {"/bin/ls", "-l", NULL};
6   char *envs[] = {NULL};
7   execve("/bin/ls", args, envs);
8 }
```

Read the Unix system manual about execve and answer the following questions. (hint: "google man execve").

1. What does execve do? [5pt]

Execve essentially executes a linux command through the C language in a script. The man page shows that it executes the program pointed to by filename. Filename must be either a binary executable or a script starting with a line of the form:

! interpreter [optional-arg]

2. What is the args parameter used for? [5pt]

Args parameter is used as an array to provide pathway to your binary you wish to execute, -l to display the list, and null to elucidate the end of a line. Essentially, to provide a pathway to your binary in which you want to execute as an array.

3. Why does the args array end with a NULL? [5pt]

It ends with NULL to explain that it's the end; integer 0 means end; NULL.

4. Create another C program which uses **execve** to print out the contents of **/etc/passwd**. (hint: use "/bin/cat"). Save your C program as **cat.cc**. Note: do **NOT** modify the original ls.cc. [15pt]

```
speech-dispatcher:x:106:29:Speech Dispatcher,,,:/var/run/speech-di
spatcher:/bin/sh
usbmux:x:107:46:usbmux daemon,,,:/home/usbmux:/bin/false
haldaemon:x:108:114:Hardware abstraction layer,,,:/var/run/hald:/b
in/false
kernoops:x:109:65534:Kernel Oops Tracking Daemon,,,:/:/bin/false
pulse:x:110:115:PulseAudio daemon,,,:/var/run/pulse:/bin/false
rtkit:x:111:117:RealtimeKit,,,:/proc:/bin/false
saned:x:112:118::/home/saned:/bin/false
hplip:x:113:7:HPLIP system user,,,:/var/run/hplip:/bin/false
gdm:x:114:120:Gnome Display Manager:/var/lib/gdm:/bin/false
csc288:x:1000:1000:csc288,,,:/home/csc288:/bin/bash
snort:x:115:123:Snort IDS:/var/log/snort:/bin/false
telnetd:x:116:124::/nonexistent:/bin/false
vboxadd:x:999:1::/var/run/vboxadd:/bin/false
jetty:x:117:125::/usr/share/jetty:/bin/false
mysql:x:118:126:MySQL Server,,,:/var/lib/mysql:/bin/false
ganglia:x:119:127:Ganglia Monitor:/var/lib/ganglia:/bin/false
bind:x:120:128::/var/cache/bind:/bin/false
sshd:x:121:65534::/var/run/sshd:/usr/sbin/nologin
xfu1010:x:1001:1010::/home/xfu1010:/bin/sh
peercast:x:122:129::/usr/share/peercast:/bin/false
csc288@csc288-laptop:~/Desktop/csc215/lectures/lec16_assembly_prac
tice$
```

5. Now compile **ls.cc** using static linking – `g++ ls.cc -o ls -static -g`. The following is the assembly of the main function. The following questions will ask you to correlate each statement in ls.cc with the assembly instructions below. You can use **GDB** to assist your analysis.

```
(gdb) x/16i $eip
=> 0x8048259 <main+9>:     movl   $0x80ae448,0x10(%esp)
   0x8048261 <main+17>:    movl   $0x80ae450,0x14(%esp)
   0x8048269 <main+25>:    movl   $0x0,0x18(%esp)
   0x8048271 <main+33>:    movl   $0x0,0x1c(%esp)
   0x8048279 <main+41>:    lea    0x1c(%esp),%eax
   0x804827d <main+45>:    mov    %eax,0x8(%esp)
   0x8048281 <main+49>:    lea    0x10(%esp),%eax
   0x8048285 <main+53>:    mov    %eax,0x4(%esp)
   0x8048289 <main+57>:    movl   $0x80ae448,(%esp)
   0x8048290 <main+64>:    call   0x805cb20 <execve>
   0x8048295 <main+69>:    mov    $0x0,%eax
   0x804829a <main+74>:    leave
   0x804829b <main+75>:    ret
```

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main(){
  char *args[] = {"/bin/ls", "-l", NULL};
  char *envs[] = {NULL};
  execve("/bin/ls", args, envs);
}
```

(5.a) In the above figure, which corresponds to the beginning address of const string "`/bin/ls`"? [10pt]

(a) 0x80ae448 (b) 0x80ae450 (c) 0x10(%esp) (d) 0x14(%esp)

(5.b) In the above figure, which corresponds to the beginning address of const string "`-l`"? [10pt]

(a) 0x80ae448 (b) 0x80ae450 (c) 0x10(%esp) (d) 0x14(%esp)

(5.c) List the assembly instructions that correspond to statement "`char *args[] = {"/bin/ls", "-l", NULL};`". [10pt]

_____0x08048259 <+9>: movl $0x80ae448,0x10(%esp)

0x08048261 <+17>: movl $0x80ae450,0x14(%esp)

0x08048269 <+25>: movl $0x0,0x18(%esp)_____

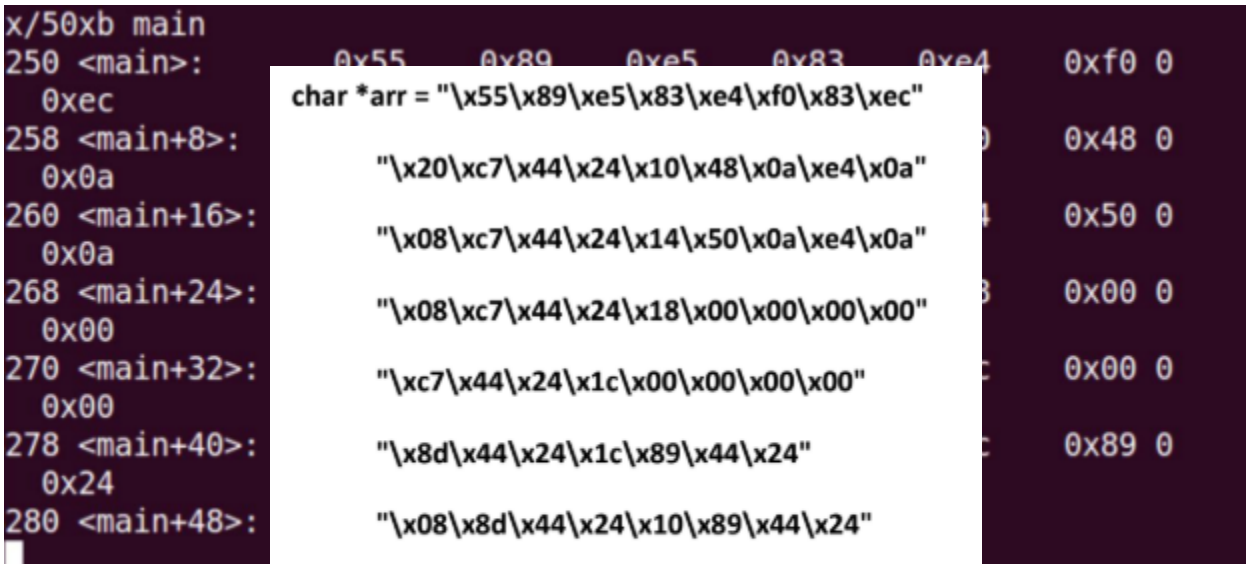
_____

_____

(5.d) Assume that the program is stopped by debugger at **0x8048261** (and the instruction `movl $0x80ae450, 0x14(%esp)` has not been executed yet). If the address of args is **0xbffff024**. What is the current value of register ESP? [10pt]

_____0xbffff014_____

(5.e) Follow the instructions in Lecture 15, and take the machine code of th e **main() function** of ls.cc and express it in the form of a C language char array (e.g., `char *arr = "\xba\x06…"`). Paste your char array below. [30pt]




```
char *arr = "\x55\x89\xe5\x83\xe4\xf0\x83\xec"
            "\x20\xc7\x44\x24\x10\x48\x0a\xe4\x0a"
            "\x08\xc7\x44\x24\x14\x50\x0a\xe4\x0a"
            "\x08\xc7\x44\x24\x18\x00\x00\x00\x00"
            "\xc7\x44\x24\x1c\x00\x00\x00\x00"
            "\x8d\x44\x24\x1c\x89\x44\x24"
            "\x08\x8d\x44\x24\x10\x89\x44\x24"
            "\x04\xc7\x04\x24\x48\xe4\x0a\x08"
            "\xe8\x8b\x48\x01\x00\xb8\x00\x00\x00"
            "\x00\xc9\xc3";
```